Documentation: Smart Lifevest

John Louis D. Lagramada

May 1, 2026

Please find the codes in the repository provided in this Github repository.

1 Software

This section outlines the embedded software architecture governing the Smart Lifevest system. The software is designed around a strictly non-blocking, interrupt-driven paradigm to ensure real-time responsiveness. It handles concurrent tasks including biological signal processing, geolocation parsing, peripheral actuation, and wireless telemetry, operating continuously to bridge the gap between physical sensor data and the external gateway network.

1.1 Lifevest Device

The Lifevest Device (Edge Transmitter) operates on an ESP32 microcontroller and acts as the primary data acquisition and local processing hub. The firmware is structured around a 3000-millisecond main telemetry loop, punctuated by a high-frequency hardware timer interrupt for precise sensor sampling. This architecture allows the device to function autonomously, evaluating localized threats (such as a spiked heart rate) while maintaining synchronized communication with the gateway.

1.1.1 Pam-Tompkins Algorithm

To accurately extract the heart rate from noisy Electrocardiogram (ECG) data, the device implements Smoothed Mode 3 of the Pan-Tompkins algorithm. The system uses an ESP32 hardware timer (`esp_timer`) to sample the analog sensor at exactly 200 Hz, placing the raw 12-bit ADC values into a 1000-sample circular buffer. The signal undergoes several digital signal processing stages within the timer callback:

- **Low-Pass Filter**: Attenuates high-frequency muscle noise and powerline interference. The difference equation implemented is:

$$y[n] = 2y[n-1] - y[n-2] + x[n-1] - 2x[n-7] + x[n-13]$$

- **High-Pass Filter**: Removes baseline wander. The code implements this using the delayed outputs of the low-pass filter buffer:

$$y[n] = y[n-1] - \frac{x[n-1]}{32} + x[n-17] - x[n-18] + \frac{x[n-33]}{32}$$

- **Derivative Filter**: Highlights the QRS complex slopes. The implementation applies a five-point derivative:

$$y[n] = \frac{1}{8}(2x[n-1] + x[n-2] - x[n-4] - 2x[n-5])$$

- **Squaring Function**: Makes all data points positive and non-linearly amplifies the higher frequency components of the QRS complex:

$$y[n] = (x[n])^2$$

- **Moving Window Integration (MWI)**: Smooths the squared signal to produce a single distinct peak for each heartbeat. The window size (N) is set to 30 samples:

$$y[n] = \frac{1}{N}\sum_{i=0}^{N-1} x[n-i]$$

- **Peak Detection & QRS Validation**: The algorithm waits for an initialization period to establish a dynamic baseline threshold (SPKI and NPKI). Detected peaks are validated against a QRS width constraint (between 0.08 and 0.20 seconds) and a refractory period (0.3 seconds) to prevent double-counting the T-wave.

- **Smoothing**: The final Beats Per Minute (BPM) is derived from the average of up to 8 recent R-R intervals. To prevent erratic jumping, the BPM change is rate-limited to a maximum delta of 1.0 BPM per second.

1.1.2 Location Tracking

Geographical coordinate parsing is handled by the TinyGPSPlus library. The ESP32 communicates with a GPS module over a dedicated hardware serial connection `HardwareSerial gpsSerial(2)` at a baud rate of 9600.

- In the main loop, `gps.encode()` continuously parses incoming NMEA sentences without blocking execution.
- The software abstracts the retrieval of coordinates into a `getActiveGps()` helper function. If the GPS lock is valid, it extracts the floating-point latitude and longitude. For the first GPS lock, the buzzer and LED actuates for three (3) short bursts.

1.1.3 Automatic LoRa & GPS Reconnection

The system relies on fault-tolerant loops to maintain connectivity:

- **LoRa Initialization**: The `attemptLoraConnection()` routine actively toggles the physical reset pin (`RST`) of the LoRa module to ensure a clean boot state before attempting to initialize communication at 433 MHz. If initialization fails, the thread traps into a non-blocking 1000-millisecond delay loop, attempting to re-establish the SPI connection indefinitely until LoRa connects.
- **GPS Reconnection & Status Tracking**: The firmware records the system timestamp every time a byte is read from the GPS module. The `updateGPS()` routine actively compares the current time against the last time the GPS received data. If the data age exceeds 5000 milliseconds, the system flags the GPS as inactive, effectively invalidating stale location data until the satellite fix is reacquired.

1.1.4 Emergency Actuation

The lifevest utilizes Pulse Width Modulation (PWM) to drive an emergency LED and a buzzer, allowing for varied intensity if needed.

- Both peripherals are attached to PWM channels operating at a frequency of 2000 Hz with an 8-bit resolution, driven at a 50% duty cycle (128 out of 255).
- Actuation is handled asynchronously via `startAsyncSignal()`, which tracks elapsed time against a target duration to toggle the pins on and off.
- Actuation can be triggered manually via a LoRa command, or automatically if it is enabled and the Pan-Tompkins algorithm reports a heart rate exceeding the heart rate threshold.

1.1.5 Non-volatile Settings

To ensure configuration persistence across reboots and battery changes, the firmware utilizes the ESP32's internal flash memory via the Preferences library.

- Upon boot, the device mounts a namespace titled "vest-settings" and loads variables into RAM, falling back to safe defaults if no prior settings exist.
- The stored settings dictate the system's core behavior, including automatic signaling rules, heart rate thresholds, actuation duration, and toggles for the LED and buzzer.
- When the device receives a specific LoRa payload (Command Code 2), it instantly updates these live variables and overwrites the Non-Volatile Storage (NVS), ensuring the new parameters remain intact permanently.

1.2 Ground Receiver Station

The Ground Receiver Station, acting as a gateway, is the critical bridge between the localized, low-power wearable device network and the wider internet. Running on a secondary ESP32 microcontroller, it listens for incoming radio signals from the lifevest and forwards that telemetry to a centralized web server. Its software architecture is built around robust network management, ensuring continuous operation even in fluctuating network conditions.

1.2.1 Automatic LoRa & WiFi Reconnection

To maintain a continuous flow of data, the gateway features independent, automated recovery mechanisms for both its radio and internet connections:

- **LoRa Hardware Recovery:** During startup or in the event of a radio module failure, the system actively reboots the physical LoRa chip by toggling its hardware reset pin. It then attempts to initialize communication on the 433 MHz band. If the chip is unresponsive, the system enters a loop, retrying once every second until a successful link is confirmed.

- **Non-blocking WiFi Watchdog:** The internet connection is monitored continuously during the system's main operational loop. If the WiFi drops, the system registers the disconnection and triggers a recovery routine. Instead of freezing the entire system while waiting for the router, it attempts to reconnect in the background once every five seconds, allowing the radio module to continue listening for emergency distress signals seamlessly.

1.2.2 HTTPS POST Request

Whenever the gateway intercepts a valid radio transmission containing the correct authentication password, it immediately translates the binary radio packet into a web-friendly JSON format.

- The payload constructed includes the device's unique identifier (`vest_id`), the user's current `heart_rate`, geographical coordinates (`latitude` and `longitude`), and critical network health metrics (radio signal strength and noise ratio).
- This data is securely forwarded to the cloud infrastructure via an HTTPS POST request.
- The system is designed to gracefully handle network timeouts, dropping the transmission if the server does not respond within 1.5 seconds, ensuring the gateway quickly returns to listening for the next radio pulse.

1.2.3 HTTPS Piggybacking

To minimize latency and network overhead, the system utilizes a "piggybacking" technique to receive instructions from the web server. Rather than the gateway constantly asking the server if there are new commands, the server attaches any pending instructions directly to its reply after receiving a telemetry update.

- Upon a successful telemetry upload, the gateway reads the server's response payload and scans it for a command code (`cmd`).
- If a command code greater than zero is detected, the system understands that the web application wants to update the lifevest's settings or trigger an alarm manually.
- To parse the incoming data without utilizing heavy, memory-intensive JSON libraries, the gateway uses a custom extraction function that hunts for specific key names (like `hr` for heart rate thresholds or `dur` for signal durations) and pulls out the associated numerical values.

1.2.4 Command Transmission and Acknowledgement

Once the gateway decodes a piggybacked command from the internet, it must ensure that the physical lifevest actually receives and executes it over the LoRa radio network. It accomplishes this through a reliable delivery and retry loop:

- **Transmission and Wait State:** The gateway packages the instructions into a new radio packet, broadcasts it to the edge device, and flags the system as awaiting an acknowledgement (`isAwaitingAck`). It simultaneously records the exact time the command was sent.
- **Retry Logic:** If three seconds elapse without a confirmation reply from the lifevest, the gateway assumes the packet was lost in the airwaves and re-transmits the command. It will attempt this up to five times (`retryCount`) before conceding that the vest is unreachable.
- **Server Notification:** If the lifevest successfully receives the command and replies with a valid acknowledgement packet, the gateway immediately cancels its retry timer. It then fires a final, brief HTTP POST request back to a dedicated server endpoint to confirm that the instruction was successfully fulfilled.

1.3 Website

The website serves as the central command and monitoring hub for the Smart Lifevest system. Hosted entirely in the cloud, it is responsible for securely receiving the relayed data from the ground receiver station, permanently storing this information, and serving a live dashboard to both end-users and system administrators.

1.3.1 Database

The system's memory is structured as a relational database, designed to efficiently link user profiles with their real-time geographical and biological data.

- **Entity Relationship Diagram**: The database architecture is divided into three primary categories:

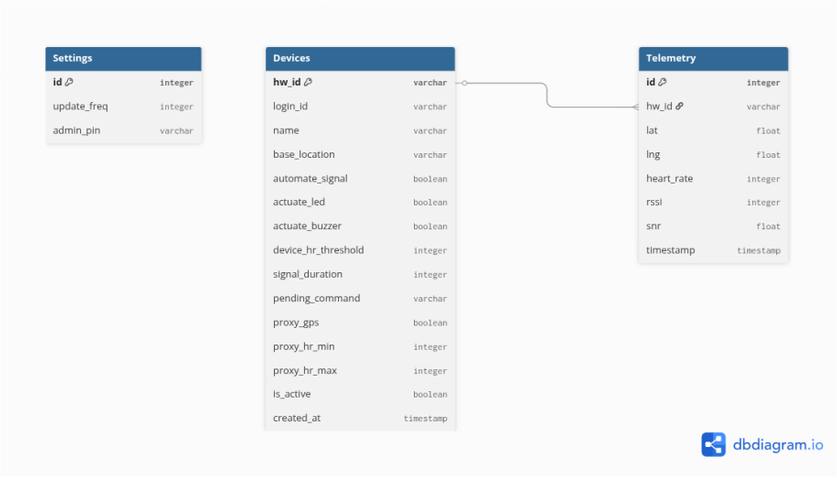

Figure 1: Entity relationship diagram.

  - The `Devices` table acts as the master registry. It bridges the physical hardware identifier (`hw_id`) to a user-facing access code (`login_id`). Beyond identification, it stores the personalized safety parameters for each vest, such as the heartbeat limit (`device_hr_threshold`) and whether the alarms should trigger on their own (`automate_signal`). It also contains a vital `pending_command` queue, which acts as a holding area for instructions waiting to be sent back down to the physical lifevest.
  - The `Telemetry` table serves as the continuous logbook. Every time the lifevest transmits a pulse, a new record is created here containing the user's `heart_rate`, exact GPS coordinates (`lat` and `lng`), radio signal quality, and a precise `timestamp`. This table is strictly tethered to the Devices table; if an administrator deletes a vest from the system, all of its historical tracking data is automatically erased as well.
  - The `Settings` table is a secure, isolated configuration file that holds global variables for the web application, such as the master password (`admin_pin`) required to access the backend dashboard.

- **Scheduled Telemetry Table Refresh**: Because the lifevest transmits data every few seconds, the database could quickly become bloated with millions of location points, slowing down the server and incurring high storage costs. To prevent this, the software includes an automated daily cleanup routine. Operating on a strict schedule (a CRON job set to run at midnight), the server sweeps through the `Telemetry` table and permanently deletes any tracking records that are older than one day. This ensures the database remains lightweight and focused only on the user's most recent activity.

1.3.2 Cloud Deployment

To guarantee that the monitoring system remains online during emergencies without the need for manual server maintenance, the backend is deployed entirely on a "serverless" cloud infrastructure.

- **Cloudflare Domains**: The cloud provider manages the domain routing, ensuring that all data—whether it is a telemetry upload from the ground station or a user checking their dashboard on a smartphone—is transmitted over a secure, encrypted HTTPS connection. This prevents sensitive geographical and heart rate data from being intercepted over public networks.

- **Cloudflare Workers**: Instead of a traditional server computer that runs constantly, the application's core logic runs on Cloudflare Workers using the Hono web framework. In this environment, the server code effectively "sleeps" until an event occurs—such as a user loading the website or the receiver station sending a `POST` request containing new data. Once triggered, the worker instantly wakes up, processes the data, and shuts back down. This architecture also securely manages access, utilizing an `adminAuth` checkpoint to intercept requests and verify the administrator's PIN before allowing any changes to the system.

- **Cloudflare D1**: This is the cloud-native SQL database system that houses the tables mentioned above. Bound directly to the serverless Worker under the name `lifevest-db`, it allows the application to instantly read and write emergency data across a globally distributed network without the delays typically associated with connecting to traditional, centralized database servers.

1.3.3 Login

- **User**: The user login interface (`index.html`) provides a simple, direct entry point for tracking a specific lifevest. Users input a unique, secure "session ID" (e.g., VEST-892A) into the

`#lifevestId` field. Upon submission, the frontend Javascript intercepts the request and verifies the ID against the backend database using the `/api/live/:login_id` endpoint. If the ID is valid and active, the user is seamlessly routed to the tracking dashboard.

- **Admin**: The administrative interface (`admin.html`) acts as the control center for the entire fleet. Access is restricted via a secure PIN overlay (`#adminAuthOverlay`). The entered PIN is verified against the backend settings database. Once authenticated, administrators can view the real-time online status of all registered hardware, assign new lifevests to login session IDs, modify global polling frequencies, and change the admin password.

1.3.4 Dashboard

The Dashboard is the primary graphical user interface for the system, accessible via a web browser on a smartphone or computer. It translates raw telemetry data from the database into an intuitive, real-time tracking and control center for the user.

- **Settings**: Users can customize the behavior of the specific lifevest they are tracking via a dedicated configuration menu.
  - This menu allows the adjustment of critical parameters, including the `hr_threshold` (the beats-per-minute limit that triggers an alarm), the `signal_duration` for the emergency peripherals, and manual toggles to enable or disable the LED and buzzer independently.
  - When saved, the web interface packages these settings and queues them in the database to be "piggybacked" to the physical lifevest during its next transmission cycle.
- **Geolocation Tracking and Waypoint**: The dashboard utilizes an interactive digital map to visualize the exact geographical coordinates of both the smartphone (the viewer) and the lifevest (the wearer).
  - A dynamically drawn dashed line visually connects the two markers on the screen.
  - To enhance situational awareness, the interface features a synchronized radar animation; it calculates the physical distance between the two points and precisely delays a flashing animation on the wearer's icon so that it illuminates exactly when the viewer's expanding radar wave "hits" it on the screen.
- **Phone Navigational Mode**: For active search-and-rescue scenarios, the user can toggle a specialized navigation mode.
  - When activated, the web application requests permission to access the smartphone's internal compass (magnetometer).
  - The software then locks the map camera to the user's location and actively rotates the entire map container inversely to the phone's physical heading. This ensures the digital map always aligns with the real world, similar to dedicated turn-by-turn GPS applications.
- **Heart Rate and Distance Display**: Real-time vital signs and spatial data are persistently displayed at the bottom of the screen.
  - The system calculates the exact distance in meters between the phone's GPS and the lifevest's GPS.
  - The heart rate monitor constantly compares the live BPM against the user-defined threshold. If the threshold is exceeded, the interface automatically triggers a "Distress Mode," turning the text red, updating the status label to "IN DISTRESS," and applying a red visual vignette overlay across the entire screen to immediately alert the viewer.
- **Lifevest Status**: A prominent status badge informs the user of the system's overall health and connectivity.
  - This is determined by evaluating the timestamp of the most recent data packet.
  - If data was received within the last 30 seconds, it displays as "LIVE". If the delay exceeds 30 seconds, it degrades to "WAITING," and if a full 60 seconds pass without a signal, the system officially marks the lifevest as "OFFLINE".
- **Connection Strength**: To help users understand the reliability of the radio link, the dashboard includes a standard four-bar cellular-style signal indicator.
  - The system dynamically fills these bars based on the Received Signal Strength Indicator (RSSI) of the LoRa packets. For example, an RSSI stronger than -95 dBm grants all four bars, while a signal dropping below -115 dBm reduces it to two.
  - If the status degrades to "OFFLINE," the bars are dimmed and a red "X" overlay is dynamically generated to clearly communicate the communication loss.
- **Signaling Activation**: A central manual override button allows the user to immediately trigger the lifevest's emergency LED and buzzer, regardless of the wearer's heart rate.

- Pressing this button queues an actuation command in the database.
- The button features "Smart ACK Logic," changing its color and text to provide real-time feedback on the radio transmission state. It turns green to indicate the command is queued for the next ping, changes to yellow (displaying "Waiting for Vest ACK...") while the radio gateway transmits it, and returns to its default blue state only when the physical lifevest confirms execution.

1.3.5 Admin Page

The Admin Page serves as the centralized command center for the organization deploying the Smart Lifevests, offering high-level control over the entire network of devices. After successfully bypassing the security overlay with the correct password, administrators are presented with a real-time overview of the active fleet and a suite of management tools.

- **Adding, Removing, Modifying Lifevests**: The platform includes a robust fleet management system to handle the physical lifecycle of the hardware:
  - **Adding**: Administrators can easily register new lifevests into the database by entering the device's physical hardware ID, assigning it a user-friendly name, logging its physical base location, and giving it an initial session ID for a user to log into.
  - **Modifying**: To accommodate maintenance, the system supports seamless hardware swapping. If a physical vest needs to be replaced, an administrator can edit the device profile and swap the old hardware ID for a new one. The backend software safely migrates all previous tracking history to the new hardware ID so the continuity of the data is never broken.
  - **Removing**: If a device is permanently retired, it can be deleted from the system. Because deleting a device also permanently erases all of its historical tracking data, the interface triggers a severe visual warning requiring explicit confirmation before the action is executed. Once authorized, both the device profile and its telemetry logs are purged from the database.
- **Global Settings**: Administrators have the authority to alter overarching parameters that dictate how the web server and the dashboards behave.
  - **Update Frequency**: This setting determines the polling rate of the system—specifically, how often (in milliseconds) the user dashboards refresh to pull the latest GPS and heart rate data from the database. Adjusting this allows administrators to balance real-time tracking responsiveness against server load.
  - **Security Management**: The global settings panel also provides the interface for changing the master administrative password (PIN) required to access the control center. When updated, the new security credentials are saved directly to the core database and take effect immediately.

2 Hardware

2.1 Lifevest Device

2.1.1 Wireless Charging

2.1.2 Waterproofing

2.1.3 Antennas

2.1.4 Boost Converters

2.1.5 MOSFET Control & Actuators

2.1.6 Sensor and Network Modules

2.1.7 Modular Velcro Attachment

2.2 Ground Receiver Station

2.2.1 LoRa Module

References

[1] Albert Einstein. "Zur Elektrodynamik bewegter Körper". In: *Annalen der Physik* (1905).